Name: Khushi Singh D20A / 30

Experiment No. 3:

Aim: Create a Cryptocurrency Using Python and Perform Mining in Blockchain

Objectives

- To understand the working of a cryptocurrency and blockchain.
- To implement a blockchain using Python.
- To create a decentralized Peer-to-Peer (P2P) blockchain network.
- To perform cryptocurrency transactions.
- To perform Proof-of-Work mining.
- To implement the longest-chain consensus mechanism.
- To remove pending transactions after they are added to a block.

Theory:

Blockchain

A blockchain is a distributed digital ledger that stores data in a sequence of blocks. Each block contains transaction information, a timestamp, a proof value, and the hash of the previous block.

The major components of a block in this experiment are:

- **Index** – Position of the block in the blockchain.
- **Timestamp** – Time at which the block was created.
- **Transactions** – Transactions included in the block.
- **Proof** – Number obtained through the Proof-of-Work process.
- **Previous Hash** – Hash of the previous block.

Each block is linked to the previous block using its hash. Therefore, changing the contents of a previous block would invalidate the subsequent blocks.

Cryptocurrency

A cryptocurrency is a digital currency that uses cryptographic techniques and blockchain technology to record transactions without requiring a central authority.

In this experiment, a simple cryptocurrency called **HadCoin** is implemented using Python.

A transaction contains:

Sender
Receiver
Amount

For example:

```
{
   "sender": "Khushi Singh",
```

```
  "receiver": "Alice",
  "amount": 50
}
```

Transactions

When a transaction is submitted, it is not immediately stored inside a block. It is first stored in the node's pending transaction list:

self.transactions

When a miner mines a new block, these pending transactions are included in the block.

Mempool

A mempool is a temporary pool containing valid transactions that have been received by a blockchain node but have not yet been included in a block.

In this implementation:

self.transactions

acts as the pending transaction pool or simplified mempool.

After the transactions are included in a block, the list is cleared using:

self.transactions = []

Thus, the same transactions are not included repeatedly in subsequent blocks.

Mining

Mining is the process of creating a new block by solving a computational Proof-of-Work problem.

The program calculates:

```
hashlib.sha256(
  str(new_proof**2 - previous_proof**2).encode()
).hexdigest()
```

The mining process continues until the generated hash starts with:

0000

The proof that satisfies this condition is used to create the new block.

Name: Khushi Singh D20A / 30

Mining Reward

The miner receives a reward for successfully mining a block.

In this implementation, the mining function adds:

```
blockchain.add_transaction(
    sender=node_address,
    receiver='Richard',
    amount=1
)
```

Therefore, a reward of **1 HadCoin** is included as a transaction in the mined block.

Multi-Node Blockchain

Three independent blockchain nodes were created:

Node 1 → Port 5001
Node 2 → Port 5002
Node 3 → Port 5003

Each node maintains its own copy of the blockchain and communicates with other nodes through HTTP requests.

Consensus and Longest Chain Rule

Different nodes may temporarily have different versions of the blockchain.

The `replace_chain()` function implements the consensus mechanism. It contacts the connected nodes, obtains their chains, checks their validity, and replaces the current chain if a longer valid chain is found.

Thus, the network eventually agrees on the longest valid blockchain.

Procedure / Steps Performed

Step 1: Download the Lab Files

The Lab 3 files were downloaded and extracted.

The following files were available:

hadcoin.py
hadcoin_node_5001.py
hadcoin_node_5002.py

hadcoin_node_5003.py
nodes.json
transaction.json

Step 2: Open the Lab 3 Directory

The Lab 3 folder was opened in PowerShell.
cd ".\Lab_3_Create a Cryptocurrency"

Step 3: Check Python Version

The Python version was checked using:
python --version
Python 3.14.7 was installed.

Step 4: Create a Virtual Environment

A virtual environment was created using:
python -m venv venv
Since PowerShell execution policy prevented direct activation, the Python executable inside the virtual environment was used directly.

Step 5: Install Required Libraries

Flask and Requests were installed using:
.\venv\Scripts\python.exe -m pip install Flask
and:
.\venv\Scripts\python.exe -m pip install requests
The installation was verified successfully.

Step 6: Run Node 5001

The first blockchain node was started using:
.\venv\Scripts\python.exe .\hadcoin_node_5001.py
The node started successfully at:
http://127.0.0.1:5001

Step 7: Run Node 5002

A second terminal was opened and Node 5002 was started:

.\venv\Scripts\python.exe .\hadcoin_node_5002.py

The node started at:

http://127.0.0.1:5002

Step 8: Run Node 5003

A third terminal was opened and Node 5003 was started:

.\venv\Scripts\python.exe .\hadcoin_node_5003.py

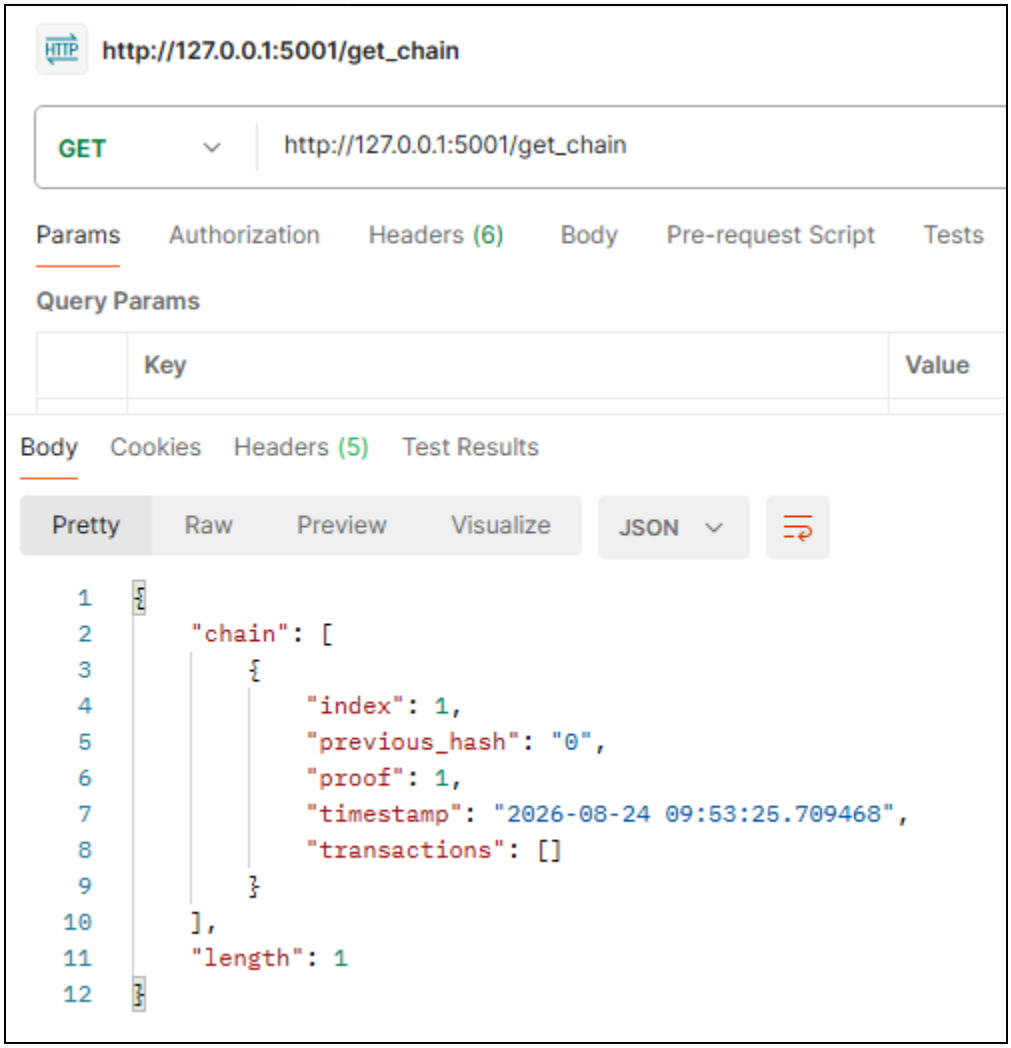
The node started at:

http://127.0.0.1:5003

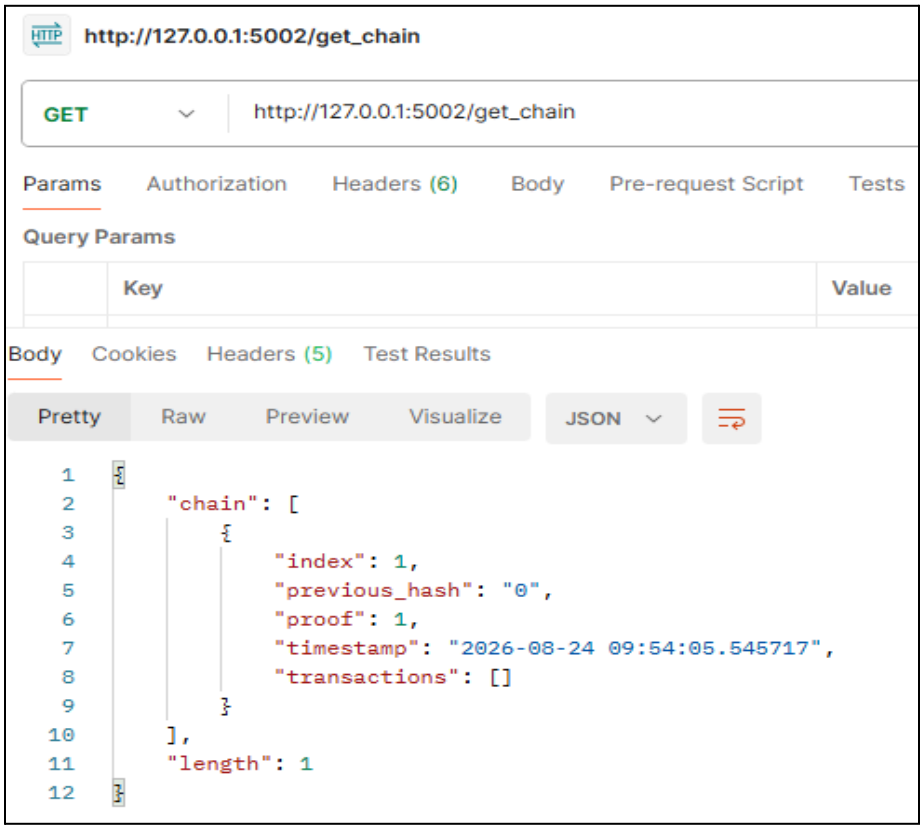
Thus, three blockchain nodes were running simultaneously.

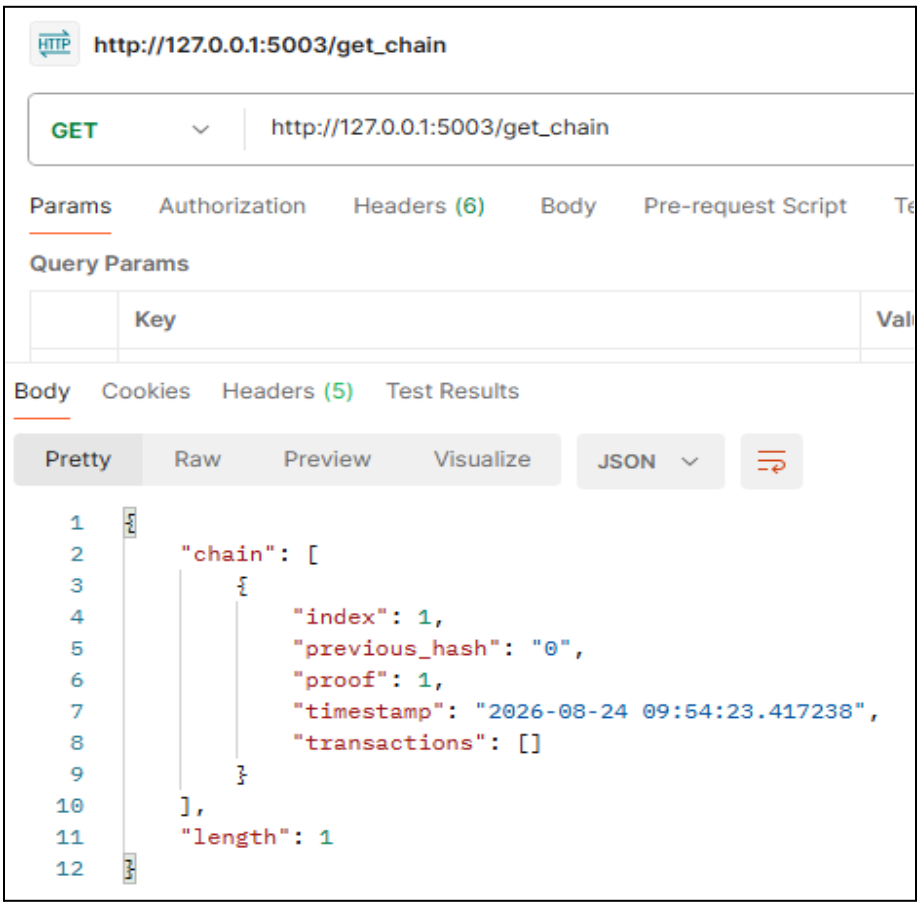
6. Testing the Blockchain Using Postman

Step 9: Fetch the Initial Blockchain



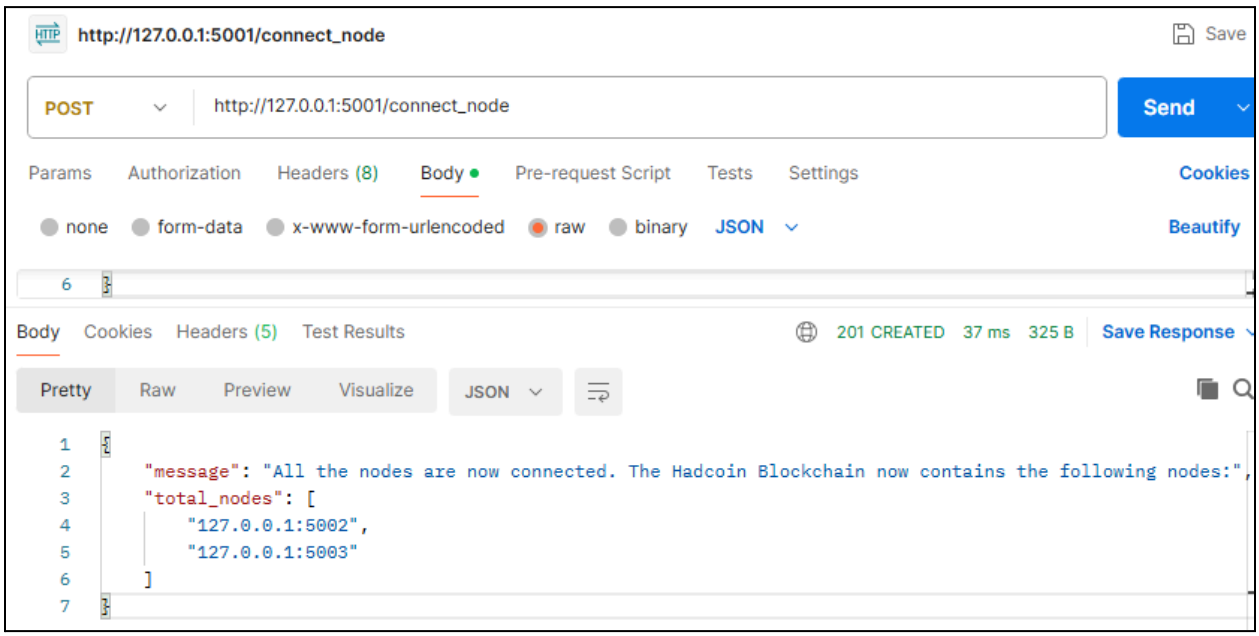
Name: Khushi Singh D20A / 30

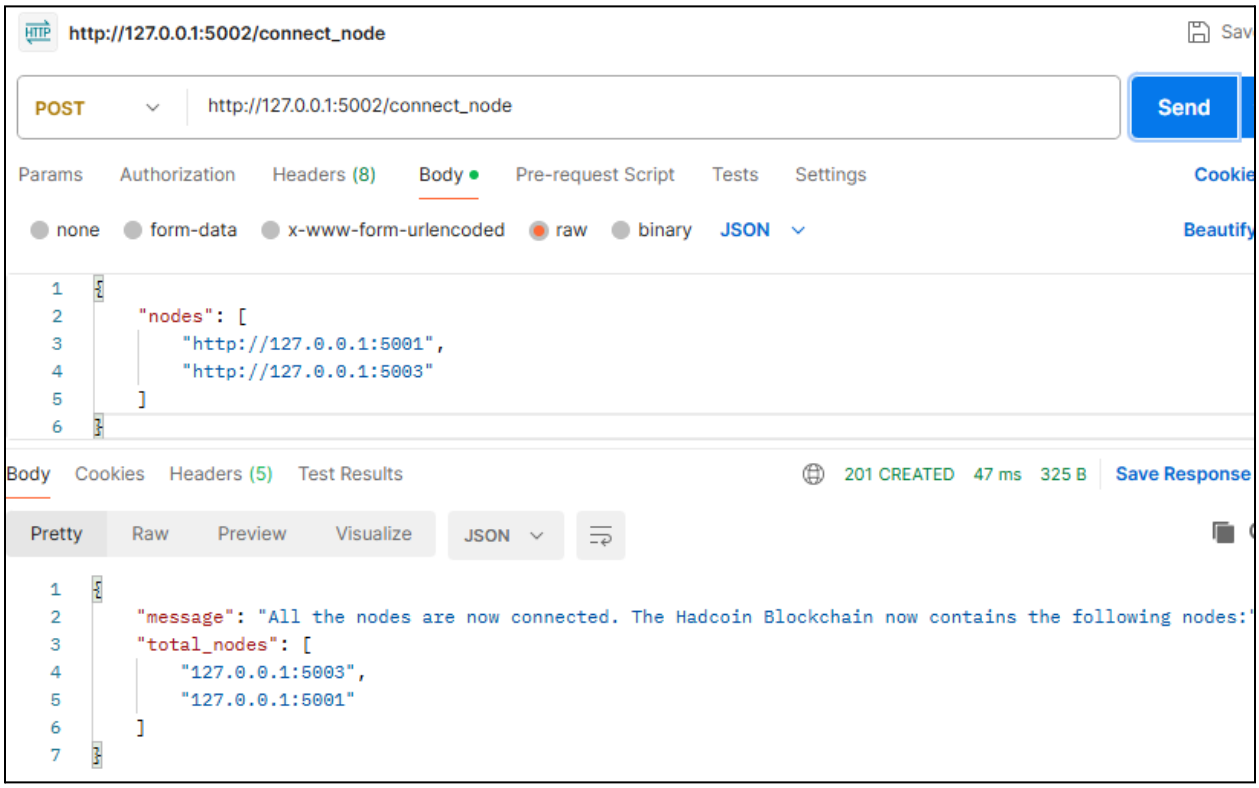




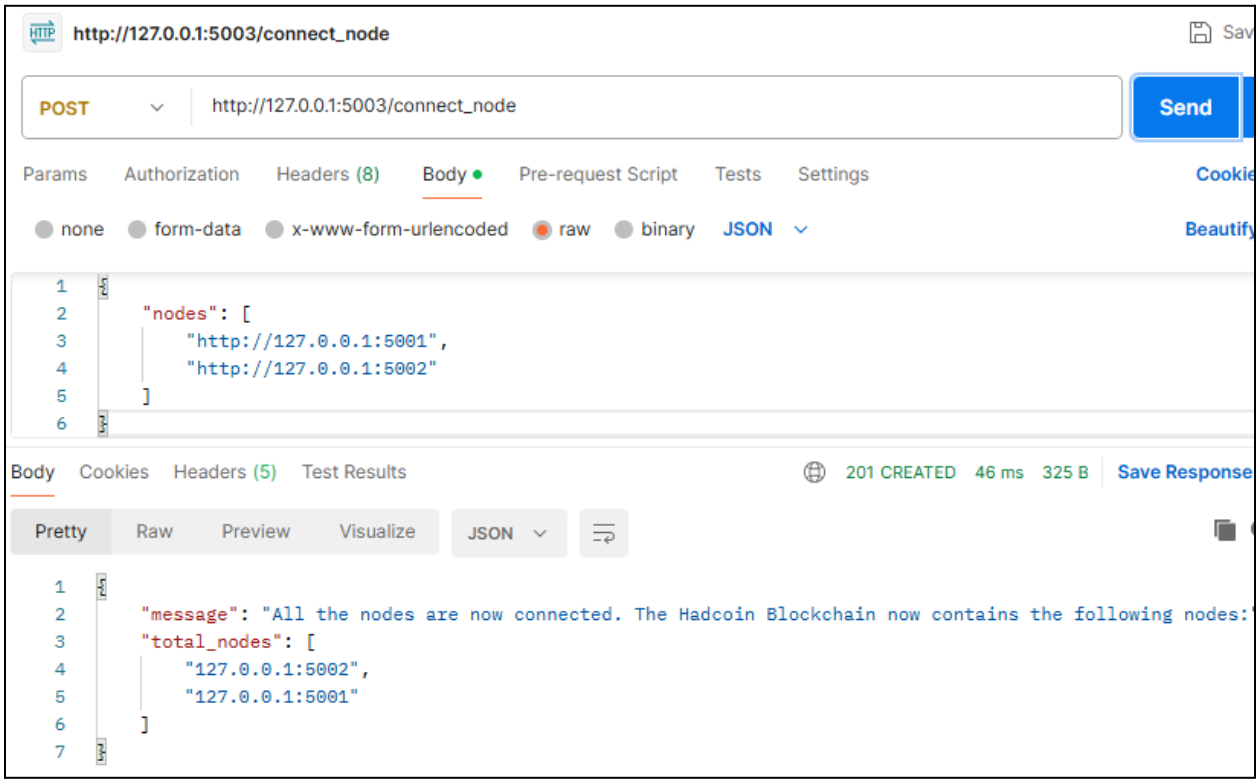
Name: Khushi Singh D20A / 30

Step 10: Connect the Nodes

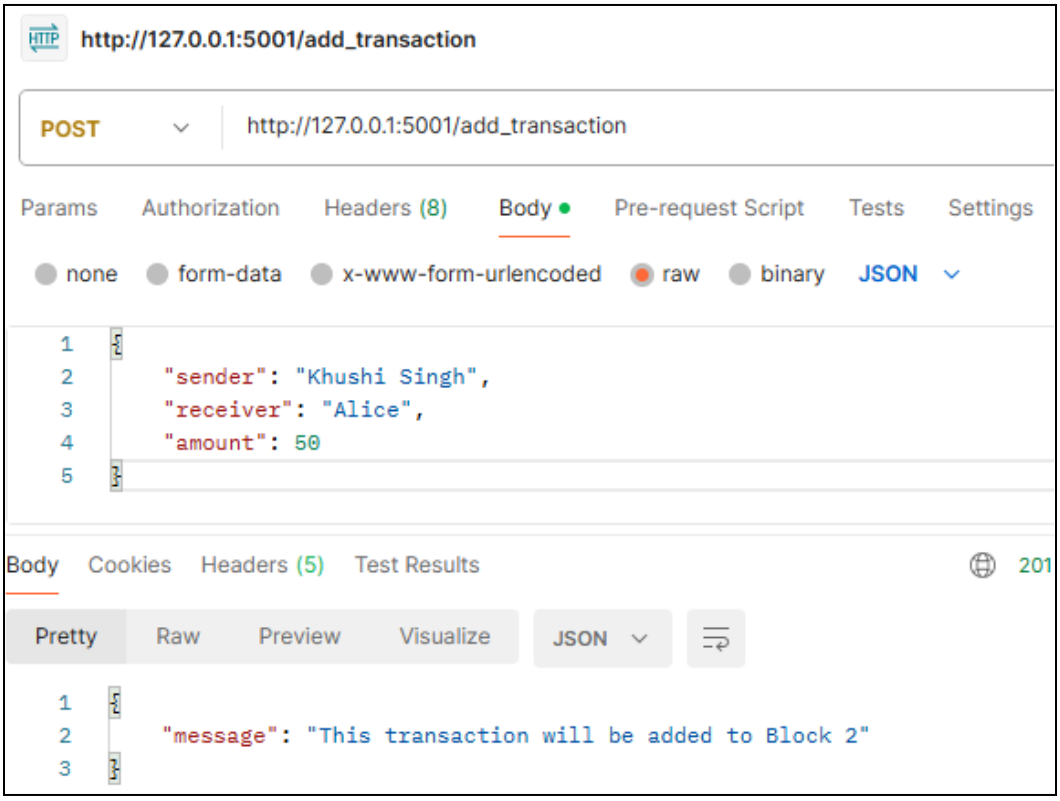




Name: Khushi Singh D20A / 30

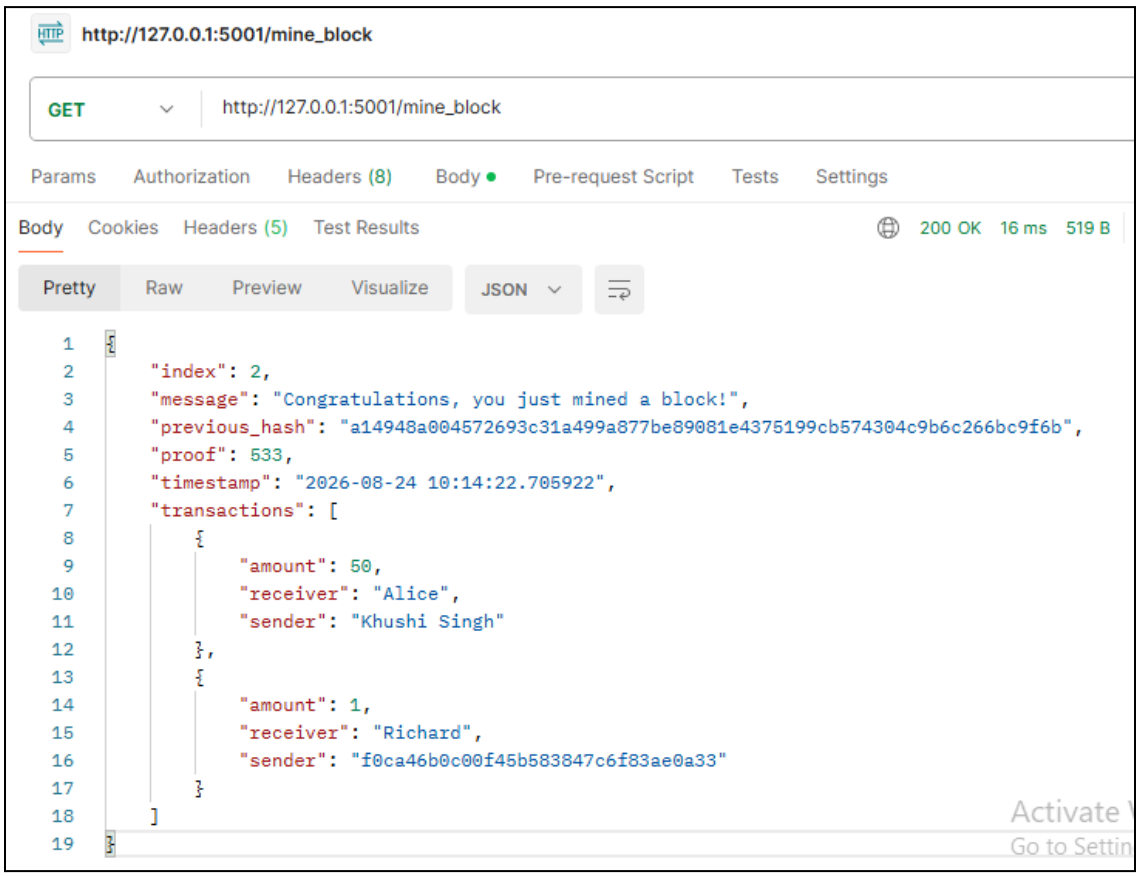


Step 11: Add a Transaction

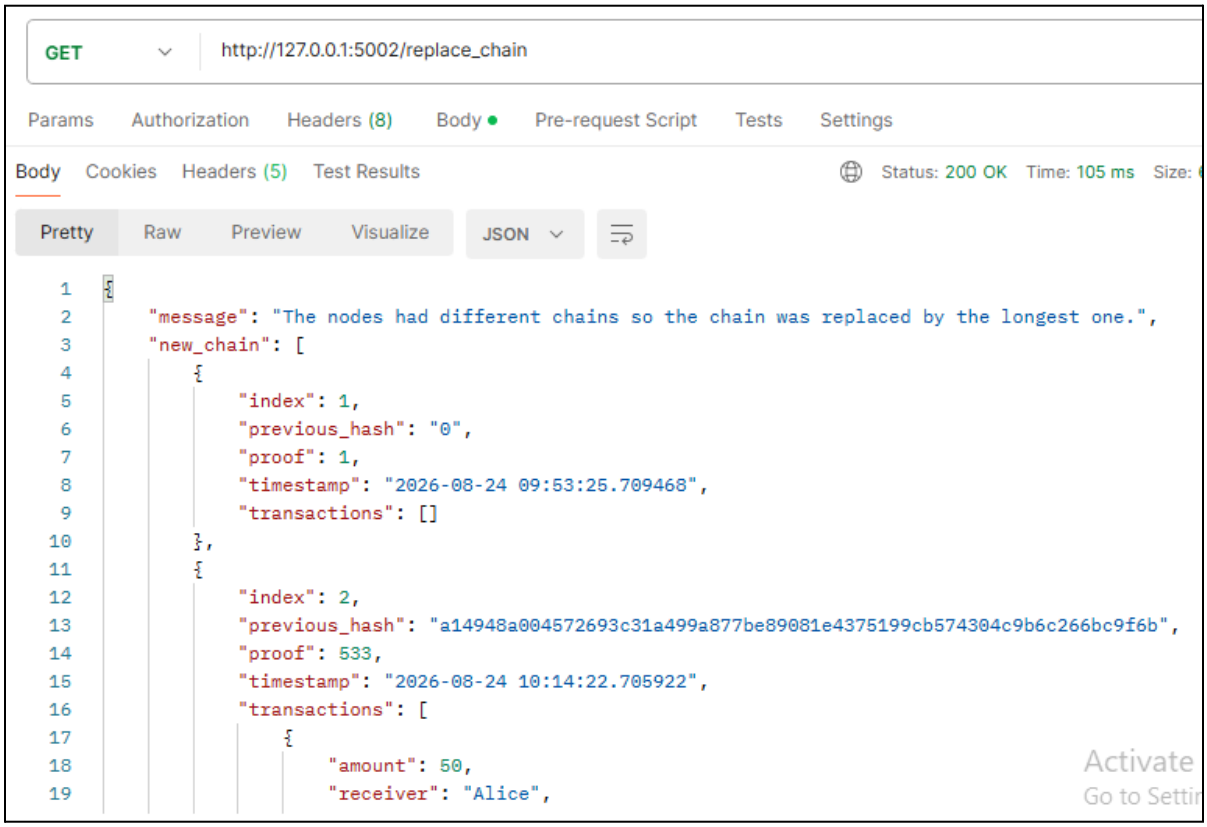


Name: Khushi Singh D20A / 30

Step 12: Mine a Block



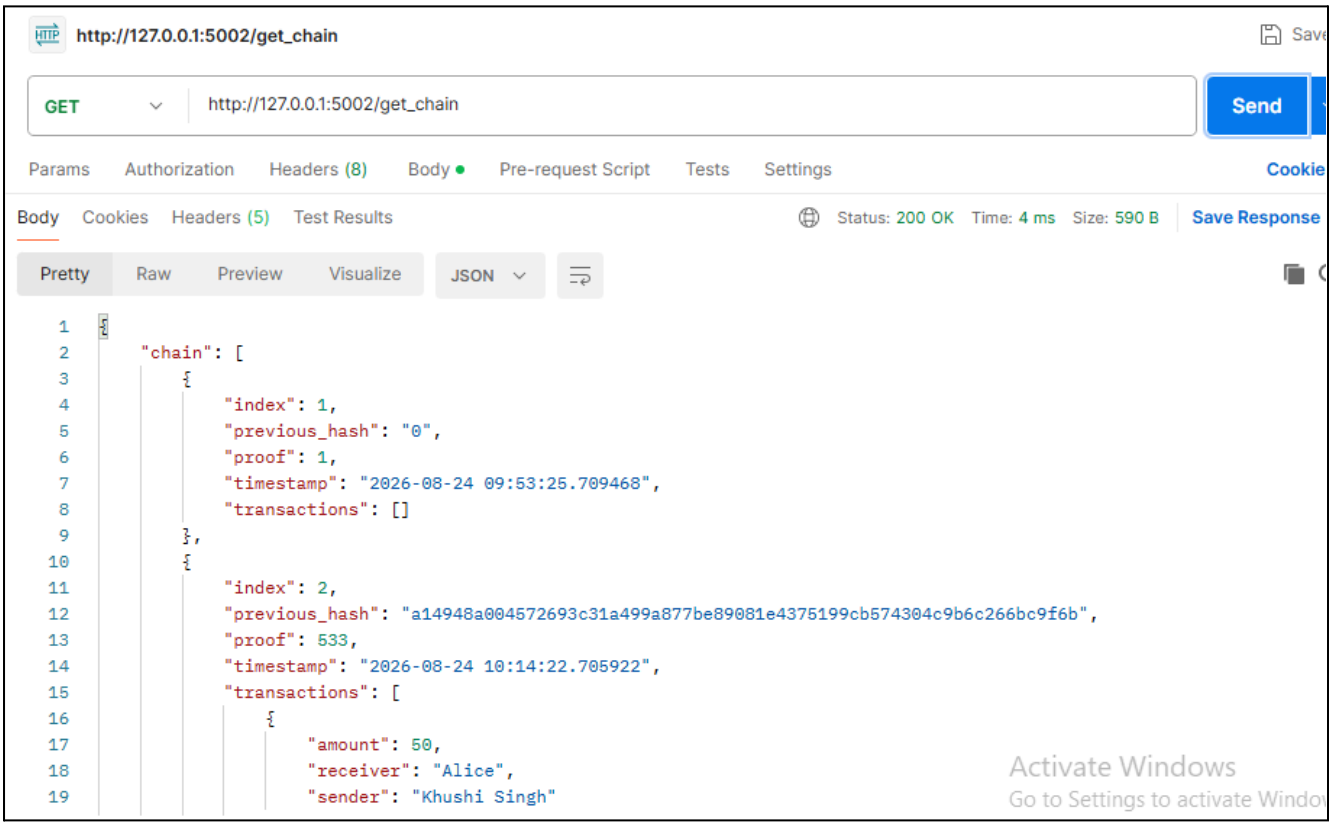
Step 13: Replace the chain using `replace_chain()`



Name: Khushi Singh D20A / 30


```
            {
                "amount": 50,
                "receiver": "Alice",
                "sender": "Khushi Singh"
            },
            {
                "amount": 1,
                "receiver": "Richard",
                "sender": "f0ca46b0c00f45b583847c6f83ae0a33"
            }
        ]
    }
  ]
}
```


Step 11: Fetch the updated chain using `get_chain()`





Step 12: Verify blockchain using `is_valid()`



Step 13: Verify that pending transactions are removed after mining

The `create_block()` function was modified/verified to clear the pending transaction list after the transactions are added to a block.

The following statement was added after assigning transactions to the block:

self.transactions = []

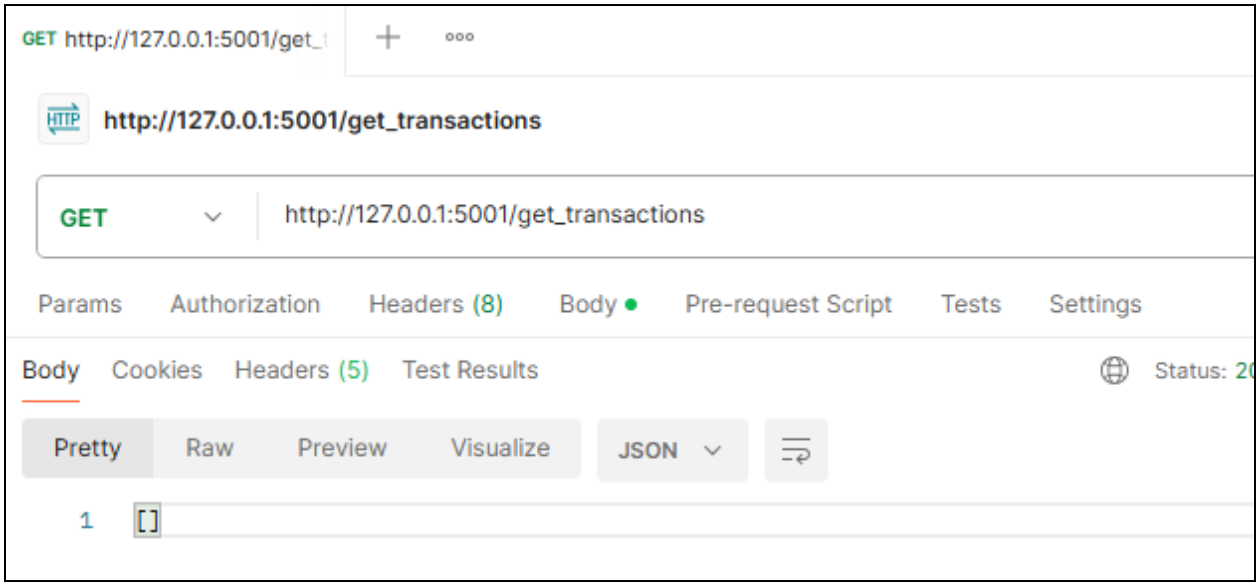
The relevant code is:

```
def create_block(self, proof, previous_hash):
    block = {
        'index': len(self.chain) + 1,
        'timestamp': str(datetime.datetime.now()),
        'proof': proof,
        'previous_hash': previous_hash,
        'transactions': self.transactions
    }

    self.transactions = []
```

```
    self.chain.append(block)
    return block
```

This ensures that once pending transactions are included in a block, they are removed from the pending transaction list and are not added again to subsequent blocks.



Conclusion

The experiment successfully demonstrated the implementation of a simple cryptocurrency and blockchain using Python and Flask. A three-node Peer-to-Peer blockchain network was created using ports 5001, 5002, and 5003. Transactions were added through Postman and stored as pending transactions before mining.

Proof-of-Work was performed to mine a new block, and the miner received a reward of 1 HadCoin. The longest-chain consensus mechanism was implemented using `replace_chain()` to synchronize nodes with the longest valid blockchain. The transaction list was also cleared after transactions were included in a block, preventing already processed transactions from being included repeatedly.

Thus, the experiment provided practical understanding of blockchain, cryptocurrency transactions, mining, Proof-of-Work, mempool, P2P communication, and consensus mechanisms.